

ПРИЛОЖЕНИЕ А

БИБЛИОТЕКА

В этой главе...

А.1. Имена и заголовки стандартной библиотеки	1077
А.2. Краткий обзор алгоритмов	1082
А.3. Случайные числа	1098

Это приложение содержит дополнительные сведения об алгоритмах и разделе случайных чисел библиотеки. В начале приведена табл. А.1, содержащая имена и заголовки стандартной библиотеки, упоминаемые в книге.

В главе 10 были использованы некоторые из наиболее популярных алгоритмов и описана архитектура, лежащая в их основе. В данном приложении перечислены все алгоритмы, упорядоченные по выполняемым ими операциям.

В разделе 17.4 (стр. 933) была описана библиотечная архитектура для случайных чисел, а также приведены примеры использования распределений нескольких типов. Библиотека определяет несколько процессоров случайного числа и двадцать распределений различных видов. В этом приложении перечислены все процессоры и распределения.

А.1. Имена и заголовки стандартной библиотеки

В программах этой книги директивы `#include`, необходимые для их компиляции, практически нигде не приводились. Для удобства читателей в табл. А.1 перечислены все использованные в программах книги библиотечные имена и заголовки, в которых они определены.

Таблица А.1. Имена и заголовки стандартной библиотеки

Имя	Заголовок
<code>abort</code>	<code><cstdlib></code>
<code>accumulate</code>	<code><numeric></code>
<code>allocator</code>	<code><memory></code>
<code>array</code>	<code><array></code>
<code>auto_ptr</code>	<code><memory></code>

Продолжение табл. А.1

Имя	Заголовок
back_inserter	<iterator>
bad_alloc	<new>
bad_array_new_length	<new>
bad_cast	<typeinfo>
begin	<iterator>
bernoulli_distribution	<random>
bind	<functional>
bitset	<bitset>
boolalpha	<iostream>
cerr	<iostream>
cin	<iostream>
cmatch	<regex>
copy	<algorithm>
count	<algorithm>
count_if	<algorithm>
cout	<iostream>
cref	<functional>
csub_match	<regex>
dec	<iostream>
default_float_engine	<iostream>
default_random_engine	<random>
deque	<deque>
domain_error	<stdexcept>
end	<iterator>
endl	<iostream>
ends	<iostream>
equal_range	<algorithm>
exception	<exception>
fill	<algorithm>
fill_n	<algorithm>
find	<algorithm>
find_end	<algorithm>
find_first_of	<algorithm>
find_if	<algorithm>
fixed	<iostream>
flush	<iostream>
for_each	<algorithm>
forward	<utility>
forward_list	<forward_list>
free	cstdlib
front_inserter	<iterator>
fstream	<fstream>

Продолжение табл. А.1

Имя	Заголовок
function	<functional>
get	<tuple>
getline	<string>
greater	<functional>
hash	<functional>
hex	<iostream>
hexfloat	<iostream>
ifstream	<fstream>
initializer_list	<initializer_list>
inserter	<iterator>
internal	<iostream>
ios_base	<ios_base>
isalpha	<cctype>
islower	<cctype>
isprint	<cctype>
ispunct	<cctype>
isspace	<cctype>
istream	<iostream>
istream_iterator	<iterator>
istringstream	<sstream>
isupper	<cctype>
left	<iostream>
less	<functional>
less_equal	<functional>
list	<list>
logic_error	<stdexcept>
lower_bound	<algorithm>
lround	<cmath>
make_move_iterator	<iterator>
make_pair	<utility>
make_shared	<memory>
make_tuple	<tuple>
malloc	cstdlib
map	<map>
max	<algorithm>
max_element	<algorithm>
mem_fn	<functional>
min	<algorithm>
move	<utility>
multimap	<map>
multiset	<set>
negate	<functional>
noboolalpha	<iostream>

Продолжение табл. А.1

Имя	Заголовок
normal_distribution	<random>
noshowbase	<iostream>
noshowpoint	<iostream>
noskipws	<iostream>
not1	<functional>
nothrow	<new>
nothrow_t	<new>
nunitbuf	<iostream>
nouppercase	<iostream>
nth_element	<algorithm>
oct	<iostream>
ofstream	<fstream>
ostream	<iostream>
ostream_iterator	<iterator>
ostreamstringstream	<sstream>
out_of_range	<stdexcept>
pair	<utility>
partial_sort	<algorithm>
placeholders	<functional>
placeholders::_1	<functional>
plus	<functional>
priority_queue	<queue>
ptrdiff_t	<cstddef>
queue	<queue>
rand	<random>
random_device	<random>
range_error	<stdexcept>
ref	<functional>
regex	<regex>
regex_constants	<regex>
regex_error	<regex>
regex_match	<regex>
regex_replace	<regex>
regex_search	<regex>
remove_pointer	<type_traits>
remove_reference	<type_traits>
replace	<algorithm>
replace_copy	<algorithm>
reverse_iterator	<iterator>
right	<iostream>
runtime_error	<stdexcept>
scientific	<iostream>
set	<set>

Продолжение табл. А.1

Имя	Заголовок
set_difference	<algorithm>
set_intersection	<algorithm>
set_union	<algorithm>
setfill	<iomanip>
setprecision	<iomanip>
setw	<iomanip>
shared_ptr	<memory>
showbase	<iostream>
showpoint	<iostream>
size_t	<cstdint>
skipws	<iostream>
smatch	<regex>
sort	<algorithm>
sqrt	<cmath>
sregex_iterator	<regex>
ssub_match	<regex>
stable_sort	<algorithm>
stack	<stack>
stoi	<string>
strcmp	<cstring>
strcpy	<cstring>
string	<string>
stringstream	<sstream>
strlen	<cstring>
strncpy	<cstring>
strtod	<string>
swap	<utility>
terminate	<exception>
time	<ctime>
tolower	<cctype>
toupper	<cctype>
transform	<algorithm>
tuple	<tuple>
tuple_element	<tuple>
tuple_size	<tuple>
type_info	<typeinfo>
unexpected	<exception>
uniform_int_distribution	<random>
uniform_real_distribution	<random>
uninitialized_copy	<memory>
uninitialized_fill	<memory>
unique	<algorithm>
unique_copy	<algorithm>

Окончание табл. А.1

Имя	Заголовок
<code>unique_ptr</code>	<code><memory></code>
<code>unitbuf</code>	<code><iostream></code>
<code>unordered_map</code>	<code><unordered_map></code>
<code>unordered_multimap</code>	<code><unordered_map></code>
<code>unordered_multiset</code>	<code><unordered_set></code>
<code>unordered_set</code>	<code><unordered_set></code>
<code>upper_bound</code>	<code><algorithm></code>
<code>uppercase</code>	<code><iostream></code>
<code>vector</code>	<code><vector></code>
<code>weak_ptr</code>	<code><memory></code>

А.2. Краткий обзор алгоритмов

В библиотеке определено более 100 алгоритмов. Чтобы научиться их использовать, следует понять структуру, а не запоминать подробности применения каждого из них. Лежащая в их основе архитектура описана в главе 10, а в этом разделе описан каждый из алгоритмов.

- `beg` и `end` — итераторы, обозначающие диапазон элементов (см. раздел 9.2.1, стр. 426). Почти все алгоритмы работают с последовательностями, обозначенными итераторами `beg` и `end`.
- `beg2` — итератор, обозначающий начало второй исходной последовательности. Если итератор `end2` присутствует, он обозначает конец второй последовательности. Если итератора `end2` нет, подразумевается, что обозначенная итератором `beg2` последовательность такого же размера, что и исходная, обозначенная итераторами `beg` и `end`. Типы итераторов `beg` и `beg2` не обязаны совпадать. Но должна существовать возможность применить указанную операцию или заданный вызываемый объект к элементам этих двух последовательностей.
- `dest` — итератор, обозначающий назначение. Последовательность назначения должна быть способна содержать столько элементов, сколько необходимо для исходной последовательности.
- `unaryPred` и `binaryPred` — унарные и бинарные предикаты (см. раздел 10.3.1, стр. 494), возвращающие применимый в условии тип и получающие соответственно один и два аргумента, являющиеся элементами исходного диапазона.
- `comp` — бинарный предикат, отвечающий требованиям упорядочивания по ключу в ассоциативном контейнере (см. раздел 11.2.2, стр. 543).

- `unaryOp` и `binaryOp` — вызываемые объекты (см. раздел 10.3.2, стр. 497), которые могут быть вызваны с одним и двумя аргументами из исходного диапазона соответственно.

А.2.1. Алгоритмы поиска объекта

Эти алгоритмы осуществляют поиск в исходной последовательности заданного значения или последовательности значений.

Каждый алгоритм предоставляет две перегруженных версии. Первая версия для сравнения элементов использует оператор равенства (`==`) базового типа, а вторая использует предоставленные пользователем предикаты `unaryPred` или `binaryPred`.

Простой алгоритм поиска

Для поиска этим алгоритмам требуются *итераторы ввода*.

```
find(beg, end, val)
find_if(beg, end, unaryPred)
find_if_not(beg, end, unaryPred)
count(beg, end, val)
count_if(beg, end, unaryPred)
```

Функция `find()` возвращает итератор на первый элемент в исходном диапазоне, равный значению `val`. Функция `find_if()` возвращает итератор на первый элемент, для которого выполняется предикат `unaryPred`. Функция `find_if_not()` возвращает итератор на первый элемент, для которого предикат `unaryPred` возвращает значение `false`. Все три функции возвращают итератор `end`, если искомый элемент не существует.

Функция `count()` возвращает количество вхождений значения `val`. Функция `count_if()` подсчитывает количество элементов, для которых предикат `unaryPred` возвращает значение `true`.

```
all_of(beg, end, unaryPred)
any_of(beg, end, unaryPred)
none_of(beg, end, unaryPred)
```

Возвращают логическое значение, указывающее, выполняется ли предикат `unaryPred` для всех элементов, какого-нибудь элемента или ни одного элемента соответственно. Если последовательность пуста, функция `any_of()` возвращает значение `false`, а функции `all_of()` и `none_of()` — `true`.[—]

Алгоритм поиска одного из нескольких значений

Этим алгоритмам требуются *прямые итераторы*. Они ищут в исходной последовательности повторяющиеся элементы.

```
adjacent_find(beg, end)
adjacent_find(beg, end, binaryPred)
```

Возвращает итератор на первую пару смежных совпадающих элементов. Возвращает итератор `end`, если смежных совпадающих элементов нет.

```
search_n(beg, end, count, val)
search_n(beg, end, count, val, binaryPred)
```

Возвращает итератор на начало внутренней последовательности из `count` равных элементов. Возвращает итератор `end`, если такой внутренней последовательности не существует.

Алгоритм поиска последовательности

За исключением алгоритма `find_first_of()` этим алгоритмам требуются две пары *прямых итераторов*. Для обозначения первой своей последовательности алгоритм `find_first_of()` использует *итераторы ввода* и *прямые итераторы* для второй. Эти алгоритмы ищут последовательность, а не одиночный элемент.

```
search(beg1, end1, beg2, end2)
search(beg1, end1, beg2, end2, binaryPred)
```

Возвращает итератор на первую позицию исходного диапазона, с которой начинается искомая последовательность. Возвращает итератор `end1`, если искомая последовательность не найдена.

```
find_first_of(beg1, end1, beg2, end2)
find_first_of(beg1, end1, beg2, end2, binaryPred)
```

Возвращает итератор на первое вхождение в первом диапазоне любого элемента из второго диапазона. Возвращает итератор `end1`, если искомое соответствие отсутствует.

```
find_end(beg1, end1, beg2, end2)
find_end(beg1, end1, beg2, end2, binaryPred)
```

Подобен алгоритму `search()`, но возвращает итератор на последнюю позицию в исходном диапазоне, в которой второй диапазон встречается как внутренняя последовательность. Возвращает итератор `end1`, если вторая последовательность пуста или не найдена.

А.2.2. Другие алгоритмы, осуществляющие только чтение

Для первых двух аргументов этим алгоритмам требуются *итераторы ввода*.

Алгоритмы `equal()` и `mismatch()` получают также дополнительный *итератор ввода*, обозначающий начало второго диапазона. Они также предоставляют две перегруженных версии. Первая версия для сравнения элементов использует оператор равенства (`==`) базового типа, а вторая сравнивает элементы используя предоставленный пользователем предикат `unaryPred` или `binaryPred`.

```
for_each(beg, end, unaryOp)
```


Вызываемый объект (см. раздел 10.3.2, стр. 497) `unaryOp` применяется к каждому элементу в исходном диапазоне. Возвращаемое значение объекта `unaryOp` (если оно есть) игнорируется. Если итераторы позволяют запись в элементы при помощи оператора обращения к значению, то вызываемый объект `unaryOp` способен изменять элементы.

```
mismatch(beg1, end1, beg2)
mismatch(beg1, end1, beg2, binaryPred)
```

Сравнивает элементы в двух последовательностях. Возвращает пару (см. раздел 11.2.3, стр. 545) итераторов, обозначающих первые элементы в каждой не совпадающей последовательности. Если все элементы соответствуют друг другу, первый итератор возвращенной пары окажется равным `end1`, а итератор `beg2` — смещению, равному размеру первой последовательности.

```
equal(beg1, end1, beg2)
equal(beg1, end1, beg2, binaryPred)
```

Выявляет равенство двух последовательностей. Возвращает значение `true`, если каждый элемент в исходном диапазоне равен соответствующему элементу последовательности, начинающейся с позиции `beg2`.

А.2.3. Алгоритмы бинарного поиска

Хотя эти алгоритмы можно использовать с прямыми итераторами, они обладают специализированными версиями, которые работают с итераторами прямого доступа и выполняются гораздо быстрее.

Этим алгоритмам требуются *прямые итераторы*, но они оптимизированы так, что выполняются намного быстрее, если вызываются с *итераторами прямого доступа*. С технической точки зрения, независимо от типа итератора, эти алгоритмы выполняют логарифмическое количество сравнений. Но при использовании с прямыми итераторами они должны выполнить линейное количество операций с итераторами для перебора элементов последовательности.

Эти алгоритмы требуют, чтобы элементы в исходной последовательности уже были упорядочены. Эти алгоритмы ведут себя подобно одноименным функциям-членам ассоциативных контейнеров (см. раздел 11.3.5, стр. 560). Алгоритмы `equal_range()`, `lower_bound()` и `upper_bound()` возвращают итераторы на позиции последовательности, куда мог бы быть вставлен заданный элемент при сохранении существующего порядка в последовательности. Если элемент больше всех остальных в последовательности, то возвращаемый итератор будет итератором после конца.

Каждый алгоритм предоставлен в двух версиях: первая использует для проверки элементов оператор меньше (`<`) типа элемента, а вторая использует заданную функцию сравнения. В следующих алгоритмах “*x меньше, чем y*” означает, что выражения `x < y` и `comp(x, y)` истинны:

```
lower_bound(beg, end, val)
lower_bound(beg, end, val, comp)
```

Возвращает итератор, обозначающий первый элемент, значение которого больше или равно значению `val`, или итератор `end`, если такого элемента нет.

```
upper_bound(beg, end, val)
upper_bound(beg, end, val, comp)
```

Возвращает итератор, обозначающий первый элемент, значение которого меньше значения `val`, или итератор `end`, если такого элемента нет.

```
equal_range(beg, end, val)
equal_range(beg, end, val, comp)
```

Возвращает пару (см. раздел 11.2.3, стр. 545), член `first` которой является итератором, возвращаемым функцией `lower_bound()`, а член `second` — итератором, возвращаемым функцией `upper_bound()`.

```
binary_search(beg, end, val)
binary_search(beg, end, val, comp)
```

Возвращает логическое значение, свидетельствующее о наличии в последовательности элемента, значение которого равно `val`. Два значения, `x` и `y`, считаются равными, если `x` не меньше `y` и `y` не меньше `x`.

А.2.4. Алгоритмы записи в элементы контейнера

Запись в элементы контейнера осуществляется многими алгоритмами. Эти алгоритмы могут отличаться видом итераторов, используемых для обозначения их исходной последовательности, а также тем, осуществляют ли они запись в элементы исходного диапазона или указанного результирующего диапазона.

Алгоритмы, которые записывают, но не читают значения элементов

Для обозначения назначения этим алгоритмам требуются *итераторы вывода*. Версии `_n` получают второй определяющий количество аргумент и записывают заданный набор элементов по назначению.

```
fill(beg, end, val)
fill_n(dest, cnt, val)
generate(beg, end, Gen)
generate_n(dest, cnt, Gen)
```

Присваивают новое значение каждому элементу исходной последовательности. Алгоритм `fill()` присваивает значение `val`; алгоритм `generate()` выполняет объект генератора `Gen`. Генератор — это вызываемый объект (см. раздел 10.3.2, стр. 497), возвращающий при каждом вызове разные значения. Алгоритмы `fill()` и `generate()` возвращают тип `void`. Версии `_n` воз-

вращают итератор на позицию непосредственно после последнего элемента, записанного в последовательность назначения.

Алгоритмы записи с итераторами ввода

Каждый из этих алгоритмов читает исходную последовательность и пишет последовательность вывода. Они требуют, чтобы *dest* был *итератором вывода*, а итераторы, обозначающие исходный диапазон, должны быть *итераторами ввода*.

```
copy(begin, end, dest)
copy_if(begin, end, dest, unaryPred)
copy_n(begin, n, dest)
```

Копирует из исходного диапазона последовательности, обозначенные итератором *dest*. Алгоритм `copy()` копирует все элементы, а алгоритм `copy_if()` копирует те из них, для которых предикат `unaryPred` истин, а алгоритм `copy_n()` копирует первые *n* элементов. У исходной последовательности должно быть по крайней мере *n* элементов.

```
move(begin, end, dest)
```

Вызов функции `std::move()` (см. раздел 13.6.1, стр. 675) для каждого элемента в исходной последовательности позволяет переместить этот элемент в последовательность, начиная с итератора *dest*.

```
transform(begin, end, dest, unaryOp)
transform(begin, end, beg2, dest, binaryOp)
```

Вызывает заданную операцию и пишет ее результат в *dest*. Первая версия применяет унарную операцию к каждому элементу в исходном диапазоне. Вторая применяет бинарную операцию к элементам этих двух исходных последовательностей.

```
replace_copy(begin, end, dest, old_val, new_val)
replace_copy_if(begin, end, dest, unaryPred, new_val)
```

Копируют каждый элемент в *dest*, заменяя определенные элементы значением *new_val*. Первая версия заменяет элементы `== old_val`, а вторая версия — элементы, удовлетворяющие предикату `unaryPred`.

```
merge(begin1, end1, beg2, end2, dest)
merge(begin1, end1, beg2, end2, dest, comp)
```

Сортирует обе исходные последовательности. Записывает в *dest* объединенную последовательность. Первая версия сравнивает элементы при помощи оператора `<`; а вторая использует предоставленный оператор сравнения.

Алгоритмы записи с прямыми итераторами

Этим алгоритмам требуются *прямые итераторы*, поскольку они пишут в элементы своих исходных последовательностей. Итераторы должны предоставлять доступ для записи в элементы.

```
iter_swap(iter1, iter2)
swap_ranges(beg1, end1, beg2)
```

Заменяет элемент, обозначенный итератором `iter1`, элементом, обозначенным итератором `iter2`; или обменивает все элементы в исходном диапазоне с таковыми из второй последовательности, начиная с позиции `beg2`. Диапазоны не должны пересекаться. Алгоритм `iter_swap()` возвращает `void`; алгоритм `swap_ranges` возвращает итератор `beg2`, увеличенный так, чтобы обозначить элемент сразу после последнего обмененного.

```
replace(beg, end, old_val, new_val)
replace_if(beg, end, unaryPred, new_val)
```

Заменяет каждый элемент, соответствующий значению `new_val`. Первая версия использует для сравнения элементов со значением `old_val` оператор `==`, а вторая заменяет те элементы, для которых истин предикат `unaryPred`.

Алгоритмы записи с двунаправленными итераторами

Поскольку этим алгоритмам необходима способность вернуться назад в последовательности, они требуют *двунаправленных итераторов*.

```
copy_backward(beg, end, dest)
move_backward(beg, end, dest)
```

Копирует или перемещает элементы из исходного диапазона в заданный. В отличие от других алгоритмов, `dest` — итератор после конца для выходной последовательности (т.е. последовательность назначения закончится непосредственно *перед* `dest`). Последний элемент в исходном диапазоне копируется или перемещается в последний элемент назначения, затем копируется (перемещается) предпоследний элемент и т.д. У элементов в последовательности назначения тот же порядок, что и в исходном диапазоне. Если диапазон пуст, возвращается итератор `dest`, в противном случае возвращается итератор на элемент, который был скопирован или перемещен из `*beg`.

```
inplace_merge(beg, mid, end)
inplace_merge(beg, mid, end, comp)
```

Объединяет две отсортированные внутренние последовательности из той же последовательности в единую, упорядоченную последовательность. Внутренние последовательности от `beg` до `mid` и от `mid` до `end` объединяются и записываются назад в первоначальную последовательность. Первая версия ис-

пользует для сравнения элементов оператор `<`, а вторая версия использует заданную операцию сравнения. Возвращают `void`.

А.2.5. Алгоритмы сортировки и разделения

Алгоритмы сортировка и разделения предоставляют различные стратегии упорядочивания элементов последовательности.

Каждый алгоритм сортировки и разделения поддерживает стабильные и нестабильные версии (см. раздел 10.3.1, стр. 496). Стабильный алгоритм обеспечивает относительный порядок равных элементов. Стабильные алгоритмы выполняют больше работы, а следовательно, могут выполняться медленней и использовать больше памяти, чем нестабильные аналоги.

Алгоритмы разделения

Алгоритмы разделения делят элементы исходного диапазона на две группы. Первая группа состоит из элементов удовлетворяющих определенному предикату, а вторая — нет. Например, элементы последовательности можно разделить на основании четности их значений или на основании того, начинается ли слово с заглавной буквы, и так далее. Этим алгоритмам требуются *двухнаправленные итераторы*.

```
is_partitioned(beg, end, unaryPred)
```

Возвращает значение `true`, если все элементы, удовлетворяющие предикату `unaryPred`, предшествуют тем, для которых предикат `unaryPred` возвращает значение `false`. Если последовательность пуста, также возвращается значение `true`.

```
partition_copy(beg, end, dest1, dest2, unaryPred)
```

Копирует в `dest1` элементы, для которых истин предикат `unaryPred`, а остальные копирует в `dest2`. Возвращает пару (см. раздел 11.2.3, стр. 545) итераторов. Член `first` пары обозначает конец скопированных в `dest1` элементов, а член `second` обозначает конец элементов, скопированных в `dest2`. Исходная последовательность не может налагаться ни на одну из результирующих последовательностей.

```
partition_point(beg, end, unaryPred)
```

Для разделения исходной последовательности используется предикат `unaryPred`. Возвращает итератор на элемент за последним, удовлетворяющим предикату `unaryPred`. Если возвращен итератор не `end`, то предикат `unaryPred` должен возвращать значение `false` для возвращенного итератора и для всех элементов, следующих за ним.

```
stable_partition(beg, end, unaryPred)  
partition(beg, end, unaryPred)
```

Для разделения исходной последовательности используется предикат `unaryPred`. Элементы, для которых истин предикат `unaryPred`, помещаются в начало последовательности, а остальные в конец. Возвращает итератор на элемент за последним, удовлетворяющим предикату `unaryPred`, или итератор `beg`, если таких элементов нет.

Алгоритмы сортировки

Этим алгоритмам требуются *итераторы прямого доступа*. Каждый из алгоритмов сортировки предоставляется в двух перегруженных версиях. В одной из них для сравнения элементов используется оператор `<` типа элемента, а во второй предусмотрен дополнительный параметр для функции сравнения (см. раздел 11.2.2, стр. 543). Алгоритм `partial_sort_copy()` возвращает итератор получателя, а остальные возвращают `void`.

Алгоритмы `partial_sort()` и `nth_element()` выполняют частичную сортировку последовательности. Их используют в случае, когда в результате сортировки всей последовательности могут возникнуть проблемы. Поскольку эти операции являются менее трудоемкими, они выполняются быстрее, чем сортировка всего исходного диапазона.

```
sort(beg, end)
stable_sort(beg, end)
sort(beg, end, comp)
stable_sort(beg, end, comp)
```

Сортирует весь диапазон.

```
is_sorted(beg, end)
is_sorted(beg, end, comp)
is_sorted_until(beg, end)
is_sorted_until(beg, end, comp)
```

Алгоритм `is_sorted()` возвращает логическое значение, указывающее, сортируется ли вся исходная последовательность. Алгоритм `is_sorted_until()` находит самую длинную изначально отсортированную часть в исходной последовательности и возвращает итератор на позицию сразу после ее конца.

```
partial_sort(beg, mid, end)
partial_sort(beg, mid, end, comp)
```

Сортирует набор элементов, количество которых равно `mid-beg`. То есть если `mid-beg` равно 42, эта функция помещает элементы с самыми низкими значениями, в отсортированном порядке, в первые 42 позиции последовательности. После завершения работы алгоритма `partial_sort()` окажутся отсортированы элементы в диапазоне от `beg` и далее, но не включая `mid`. Ни один из элементов в отсортированном диапазоне не больше, чем любой из

элементов в диапазоне после `mid`. Порядок неотсортированных элементов не определен.

```
partial_sort_copy(beg, end, destBeg, destEnd)
partial_sort_copy(beg, end, destBeg, destEnd, comp)
```

Сортирует элементы исходного диапазона и помещает их (в отсортированном порядке) в последовательность, указанную итераторами `destBeg` и `destEnd`. Если получающий диапазон имеет тот же размер или превосходит исходный, в него сохраняется весь исходный диапазон в отсортированном виде, начиная с позиции `destBeg`. Если размер получающего диапазона меньше, в него будет скопировано столько отсортированных элементов, сколько поместится.

Алгоритм возвращает итератор в получающем диапазоне, указывающий на следующий элемент после последнего отсортированного. Если получающая последовательность меньше исходного диапазона или равна ему по размеру, будет возвращен итератор `destEnd`.

```
nth_element(beg, nth, end)
nth_element(beg, nth, end, comp)
```

Аргумент `nth` должен быть итератором, указывающим на элемент в исходной последовательности. Обозначенный этим итератором элемент после выполнения алгоритма `nth_element` имеет значение, которое находилось бы там после сортировки всей последовательности. Кроме того, элементы контейнера вокруг позиции `nth` также отсортированы: перед ней располагается значение меньше или равное значению в позиции `nth`, а после нее значение, большее или равное.

А.2.6. Общие функции изменения порядка

Некоторые алгоритмы переупорядочивают элементы исходной последовательности. Первые два, `remove()` и `unique()`, переупорядочивают последовательность так, чтобы элементы в первой части удовлетворяли некоему критерию. Они возвращают итератор, отмечающий конец этой подпоследовательности. Другие, например `reverse()`, `rotate()` и `random_shuffle()`, реорганизуют всю последовательность.

Базовые версии этих алгоритмов работают “на месте”, т.е. они реорганизуют элементы непосредственно исходной последовательности. Три алгоритма изменения порядка предоставляют копирующие версии. Они записывают переупорядоченные значения в получающую последовательность, а не непосредственно в исходную. Для получающей последовательности этим алгоритмам требуются *итераторы вывода*.

Переупорядочивающие алгоритмы, использующие прямые итераторы

Эти алгоритмы переупорядочивают исходную последовательность. Им необходимы по крайней мере *прямые итераторы*.

```
remove(beg, end, val)
remove_if(beg, end, unaryPred)
remove_copy(beg, end, dest, val)
remove_copy_if(beg, end, dest, unaryPred)
```

“Удаляет” элементы из последовательности, записывая поверх них те элементы, которые должны быть сохранены. Удаляются те элементы, которые равны значению `val` или те, для которых предикат `unaryPred` вернул значение `true`. Возвращает итератор на следующий элемент после последнего удаленного.

```
unique(beg, end)
unique(beg, end, binaryPred)
unique_copy(beg, end, dest)
unique_copy_if(beg, end, dest, binaryPred)
```

Переупорядочивает последовательность так, чтобы смежные совпадающие элементы были удалены при перезаписи. Возвращает итератор на следующий элемент после последнего уникального. Для проверки совпадения двух смежных элементов первая версия использует оператор `==`, а вторая — предикат.

```
rotate(beg, mid, end)
rotate_copy(beg, mid, end, dest)
```

“Поворачивает” элементы вокруг элемента, обозначенного итератором `mid`. Элемент, указанный итератором `mid`, становится первым элементом, затем идет последовательность от `mid+1` до `end` (но не включая его), далее следует диапазон от `beg` до `mid` (но не включая его). Возвращает итератор, обозначающий элемент, который первоначально был в `beg`.

Переупорядочивающие алгоритмы, использующие двунаправленные итераторы

Поскольку эти алгоритмы обрабатывают исходную последовательность в обратном порядке, им необходимы *двунаправленные итераторы*.

```
reverse(beg, end)
reverse_copy(beg, end, dest)
```

Меняет порядок элементов последовательности на обратный. Алгоритм `reverse()` возвращает тип `void`, а алгоритм `reverse_copy()` возвращает итератор, принимающей последовательности на элемент, который расположен за последним скопированным.

Переупорядочение алгоритмов с помощью итераторов прямого доступа

Поскольку эти алгоритмы реорганизуют элементы в произвольном порядке, им нужны *итераторы прямого доступа*.

```
random_shuffle(beg, end)
random_shuffle(beg, end, rand)
shuffle(beg, end, Uniform_rand)
```

Осуществляет перестановку элементов исходной последовательности в случайном порядке. Перетасовывает элементы в исходной последовательности. Вторая версия получает вызываемый объект, получающий положительное целочисленное значение и возвращающий случайное целое число в диапазоне от нуля до заданного значения с равномерным распределением. Третий аргумент должен отвечать требованиям равномерного генератора случайных чисел (см. раздел 17.4, стр. 933). Все три версии возвращают тип `void`.

А.2.7. Алгоритмы перестановки

Алгоритмы перестановки осуществляют лексикографические перестановки последовательности. Эти алгоритмы переупорядочивают элементы так, чтобы получить лексикографически следующую или предыдущую перестановку заданной последовательности. Они возвращают логическое значение, означающее, была ли осуществлена следующая или предыдущая перестановка.

Чтобы лучше понять смысл следующей или предыдущей лексикографической перестановки, рассмотрим такую последовательность из трех символов: `abc`. У этой последовательности есть шесть возможных вариантов перестановки: `abc`, `acb`, `bac`, `bca`, `cab` и `cba`. Эти варианты перестановки перечислены в лексикографическом порядке на основании оператора “меньше”. Таким образом, вариант перестановки `abc` будет первым, поскольку его первый элемент меньше или равен первому элементу любого другого варианта перестановки, а ее второй элемент меньше, чем у любого другого варианта с тем же первым элементом. Точно так же `acb` — следующий вариант перестановки, поскольку он начинается с символа `a`, который меньше первого элемента любого из остальных вариантов перестановки. Варианты перестановки, начинающиеся с `b`, располагаются перед такими, начинающимися с `c`.

Для каждого описанного выше варианта перестановки можно выяснить, какой из них должен располагаться прежде, а какие после него. Например, о варианте перестановки `bca` можно сказать, что предыдущим для нее будет вариант `bac`, а следующим — `cab`. Для варианта `abc` нет предыдущего, а для варианта `cba` — последующего варианта перестановки.

Эти алгоритмы подразумевают, что элементы в последовательности уникальны. Таким образом, алгоритмы подразумевают, что никакие два элемента последовательности не имеют одинакового значения.

Для осуществления перестановки нужна возможность перебора последовательности вперед и назад, поэтому им требуются *двунаправленные итераторы*.

```
is_permutation(beg1, end1, beg2)
is_permutation(beg1, end1, beg2, binaryPred)
```

Алгоритмы возвращают значение `true`, если во второй последовательности есть вариант перестановки с тем же набором элементов, что и в первой последовательности, для которой элементы в варианте перестановки и в исходной последовательности равны. Первая версия сравнивает элементы, используя оператор `==`; вторая использует заданный предикат `binaryPred`.

```
next_permutation(beg, end)
next_permutation(beg, end, comp)
```

Если последовательность уже находится в последнем варианте перестановки, алгоритм `next_permutation()` переупорядочивает последовательность так, чтобы она соответствовала самой младшей версии, и возвращает значение `false`. В противном случае последовательность преобразуется в следующий вариант перестановки и возвращает значение `true`. Первая версия использует для сравнения элементов оператор `<` типа элемента, а вторая — указанную функцию сравнения.

```
prev_permutation(beg, end)
prev_permutation(beg, end, comp)
```

Подобен алгоритму `next_permutation()`, но преобразует последовательность в предыдущую версию перестановки. Если текущая версия является самой младшей, переупорядочивает последовательность в самую старшую и возвращает значение `false`.

А.2.8. Алгоритмы набора для отсортированных последовательностей

Алгоритмы набора реализуют присущие набору операции, применяемые для отсортированной последовательности. Не следует путать эти алгоритмы с функциями библиотечного контейнера `set` (набор). Они обеспечивают присущее набору поведение на базе обычного последовательного контейнера (например, `vector`, `list` и т.д.) или другой последовательности (например, потока ввода).

Поскольку эти алгоритмы обрабатывают элементы последовательно, им требуются *итераторы ввода*. За исключением алгоритма `includes` всем им необхо-

дим *итератор вывода*. Алгоритмы возвращают итератор `dest`, увеличенный так, чтобы указывать на следующий элемент после последнего записанного.

Каждый алгоритм предоставлен в двух формах: использующей для сравнения элементов оператор `<` или функцию сравнения.

```
includes(beg, end, beg2, end2)
includes(beg, end, beg2, end2, comp)
```

Возвращает значение `true`, если каждый элемент во второй последовательности содержится в исходной последовательности. В противном случае возвращает значение `false`.

```
set_union(beg, end, beg2, end2, dest)
set_union(beg, end, beg2, end2, dest, comp)
```

Создает сортируемую последовательность элементов, которые находятся в обеих последовательностях. Элементы, которые находятся в обеих последовательностях, записываются в указанную итератором `dest` результирующую последовательность в одном экземпляре.

```
set_intersection(beg, end, beg2, end2, dest)
set_intersection(beg, end, beg2, end2, dest, comp)
```

Создает сортируемую последовательность элементов, представленных в обеих последовательностях. Результат сохраняется в последовательности, указанной итератором `dest`.

```
set_difference(beg, end, beg2, end2, dest)
set_difference(beg, end, beg2, end2, dest, comp)
```

Создает сортируемую последовательность элементов, представленных в первом контейнере, но не во втором.

```
set_symmetric_difference(beg, end, beg2, end2, dest)
set_symmetric_difference(beg, end, beg2, end2, dest, comp)
```

Создает сортируемую последовательность элементов, представленных в любом из контейнеров, но не в обоих контейнерах.

А.2.9. Минимальные и максимальные значения

Эти алгоритмы используют при сравнении либо оператор `<` для типа элемента, либо заданную функцию сравнения. Алгоритмы первой группы работают со значениями, а не с последовательностями. Алгоритмы второй группы получают последовательность, обозначенную *итераторами ввода*.

```
min(val1, val2)
min(val1, val2, comp)
min(init_list)
min(init_list, comp)
max(val1, val2)
```

```
max(vall, val2, comp)
max(init_list)
max(init_list, comp)
```

Эти алгоритмы возвращают минимум или максимум значений `vall` и `val2` либо значений из списка `initializer_list`. Тип аргументов должен точно совпадать. Аргументы и тип возвращаемого значения являются ссылками на константы, а значит, объекты не копируются.

```
minmax(vall, val2)
minmax(vall, val2, comp)
minmax(init_list)
minmax(init_list, comp)
```

Возвращают пару (см. раздел 11.2.3, стр. 545), член `first` которой содержит меньшее из предоставленных значений, а член `second` — большее. Версия со списком `initializer_list` возвращает пару, член `first` которой содержит наименьшее значение в списке, а `second` — наибольшее.

```
min_element(beg, end)
min_element(beg, end, comp)
max_element(beg, end)
max_element(beg, end, comp)
minmax_element(beg, end)
minmax_element(beg, end, comp)
```

Алгоритмы `min_element()` и `max_element()` возвращают итераторы на наименьший и наибольший элементы в исходной последовательности соответственно. Алгоритм `minmax_element` возвращает пару, член `first` которой содержит наименьший элемент, а член `second` — наибольший.

Лексикографическое сравнение

Этот алгоритм сравнивает две последовательности в поисках первой неравной пары элементов. Используется либо оператор `<` типа элемента, либо заданная функция сравнения. Обе последовательности обозначаются *итераторами ввода*.

```
lexicographical_compare(beg1, end1, beg2, end2)
lexicographical_compare(beg1, end1, beg2, end2, comp)
```

Алгоритм возвращает значение `true`, если первая последовательность лексикографически меньше второй. В противном случае возвращается значение `false`. Если одна последовательность короче второй и все ее элементы совпадают с соответствующими элементами более длинной последовательности, то более короткая последовательность лексикографически меньше. Если размер последовательностей совпадает и совпадают соответствующие элементы, то ни одна из них лексикографически не меньше другой.

А.2.10. Числовые алгоритмы

Числовые алгоритмы определены в заголовке `numeric`. Этим алгоритмам требуются *итераторы ввода*; если алгоритм осуществляет запись в вывод, он использует *итератор вывода* для получателя.

```
accumulate(beg, end, init)
accumulate(beg, end, init, binaryOp)
```

Возвращает сумму всех значений в исходном диапазоне. Суммирование начинается с исходного значения, заданного параметром `init`. Тип возвращаемого значения задает тип параметра `init`. Первая версия использует оператор `+` типа элемента, а вторая — указанный бинарный оператор.

```
inner_product(beg1, end1, beg2, init)
inner_product(beg1, end1, beg2, init, binOp1, binOp2)
```

Возвращает сумму элементов, полученных как произведение двух последовательностей. Обе последовательности обрабатываются совместно и элементы из каждой последовательности умножаются. Результат умножения суммируется. Исходное значение суммы определяет `init`. Тип `init` определяет тип возвращаемого значения.

Первая версия использует операторы умножения (`*`) и сложения (`+`) элементов. Вторая версия применяет заданные бинарные операторы, используя первый оператор вместо суммы и второй вместо умножения.

```
partial_sum(beg, end, dest)
partial_sum(beg, end, dest, binaryOp)
```

Пишет в `dest` новую последовательность, каждое значение элемента которой представляет собой сумму всех предыдущих элементов до (и включая) своей позиции в пределах исходного диапазона. Первая версия использует оператор `+` типа элемента, а вторая — заданный бинарный оператор. Возвращает итератор `dest`, увеличенный так, чтобы указывать на следующий элемент после последнего записанного.

```
adjacent_difference(beg, end, dest)
adjacent_difference(beg, end, dest, binaryOp)
```

Пишет в `dest` новую последовательность, каждое значение элемента которой, кроме первого, представляет собой разницу между текущими и предыдущим элементами. Первая версия использует оператор `-` типа элемента, а вторая применяет заданный бинарный оператор.

```
iota(beg, end, val)
```

Присваивает `val` первому элементу и осуществляет приращение `val`. Присваивает приращенное значение следующему элементу и снова осуществляет приращение `val`, а затем присваивает приращенное значение следующему

элементу последовательности. Продолжает приращение `val` и присваивает новое значение последующему элементу в исходной последовательности.

А.3. Случайные числа

Библиотека определяет набор классов процессоров случайных чисел и адаптеров, использующих различные математические подходы для генерации псевдослучайных чисел. Библиотека определяет также набор шаблонов распределений, обеспечивающих распределение чисел согласно различным вероятностям. Имена классов процессоров и распределений соответствуют их математическим свойствам.

Подробности генерации чисел этими классами не рассматриваются в данном издании. Здесь перечислены типы процессоров и распределений, но чтобы лучше узнать, как их использовать, следует обратиться к другим ресурсам.

А.3.1. Распределение случайных чисел

За исключением распределения `bernouilli_distribution`, всегда генерирующего логические значения, типы распределений являются шаблонами. Каждый из этих шаблонов получает один параметр типа, задающий тип генерируемого распределением результата.

Классы распределений отличаются от других использованных ранее шаблонов класса, поскольку типы распределения налагают ограничения на пригодные для использования типы. Некоторые шаблоны распределения применяются для генерации только чисел с плавающей запятой; другие применяются для генерации только целых чисел.

В описаниях ниже для указания типа генерируемых шаблоном распределения чисел, например с плавающей запятой, используется формат *имя_шаблона*<RealT>. Для таких шаблонов вместо RealT можно использовать типы `float`, `double` или `long double`. Точно так же вместо IntT можно использовать любой из встроенных целочисленных типов (`short`, `int`, `long`, `long long`, `unsigned short`, `unsigned int`, `unsigned long` или `unsigned long long`), но не тип `bool` или `char`.

Шаблоны распределения определяют заданный по умолчанию параметр типа шаблона (см. раздел 17.4.2, стр. 940). Для целочисленных распределений по умолчанию принят тип `int`; для распределений, генерирующих числа с плавающей запятой, — тип `double`.

Конструкторы каждого вида распределения имеют специфические параметры. Некоторые из этих параметров определяют диапазон распределения. В отличие от диапазонов итераторов, эти диапазоны всегда являются инклюзивными (включающими крайние значения).

Равномерное распределение

```
uniform_int_distribution<IntT> u(m, n);  
uniform_real_distribution<RealT> u(x, y);
```

Генерирует значения указанного типа в заданном инклюзивном диапазоне. Параметры *m* (или *x*) задают наименьшее число, которое может быть возвращено; а параметры *n* (или *y*) — наибольшее. По умолчанию *m* имеет значение 0, а *n* — максимально возможное значение, которое способен хранить объект типа *IntT*. Параметр *x* по умолчанию имеет значение 0.0, а *y* — 1.0.

Распределение Бернулли

```
bernoulli_distribution b(p);
```

Возвращает значение *true* с вероятностью, заданной параметром *p*. По умолчанию параметр *p* имеет значение 0.5.

```
binomial_distribution<IntT> b(t, p);
```

Распределение вычисляется для выборочного размера, заданного целочисленным значением *t*, с вероятностью *p*; по умолчанию *t* имеет значение 1, а *p* — значение 0.5.

```
geometric_distribution<IntT> g(p);
```

Параметр *p* задает вероятность возвращения значения *true* и по умолчанию имеет значение 0.5.

```
negative_binomial_distribution<IntT> nb(k, p);
```

Целочисленное значение *k* приближается к решению с вероятностью успеха *p*. По умолчанию *k* имеет значение 1, а *p* — значение 0.5.

Распределение Пуассона

```
poisson_distribution<IntT> p(x);
```

Распределение относительно значения *x* типа *double*.

```
exponential_distribution<RealT> e(lam);
```

Лямбда *lam* — значение с плавающей точкой; по умолчанию *lam* имеет значение 1.0.

```
gamma_distribution<RealT> g(a, b);
```

Альфа (форма) *a* и бета (масштаб) *b*; оба по умолчанию имеют значение 1.0.

```
weibull_distribution<RealT> w(a, b);
```

Форма *a* и масштаб *b*; оба по умолчанию имеют значение 1.0.

```
extreme_value_distribution<RealT> e(a, b);
```

По умолчанию *a* имеет значение 0.0, а *b* — значение 1.0.

Нормальное распределение или распределение Гаусса

```
normal_distribution<RealT> n(m, s);
```

Параметр m — это математическое ожидание, а s — среднеквадратичное отклонение. По умолчанию m имеет значение 0.0, а s — значение 1.0.

```
lognormal_distribution<RealT> ln(m, s);
```

Параметр m — это математическое ожидание, а s — среднеквадратичное отклонение. По умолчанию m имеет значение 0.0, а s — значение 1.0.

```
chi_squared_distribution<RealT> c(x);
```

Параметр x — это степень свободы; по умолчанию имеет значение 1.0.

```
cauchy_distribution<RealT> c(a, b);
```

Область a по умолчанию имеет значение 0.0, а масштаб b — значение 1.0.

```
fisher_f_distribution<RealT> f(m, n);
```

m и n — степени свободы; оба по умолчанию имеют значения 1.

```
student_t_distribution<RealT> s(n);
```

n — степень свободы; значение по умолчанию — 1.

Выборочное распределение

```
discrete_distribution<IntT> d(i, j);
```

```
discrete_distribution<IntT> d(il);
```

i и j — итераторы ввода последовательности коэффициентов; il — заключенный в скобки список коэффициентов. Коэффициенты должны допускать приведение к типу `double`.

```
piecewise_constant_distribution<RealT> pc(b, e, w);
```

b , e и w — итераторы ввода.

```
piecewise_linear_distribution<RealT> pl(b, e, w);
```

b , e и w — итераторы ввода.

А.3.2. Процессоры случайных чисел

Библиотека определяет три класса, реализующих различные алгоритмы генерации случайных чисел. Библиотека определяет также три адаптера, модифицирующих созданную последовательность заданным процессором. Классы процессоров и адаптеров являются шаблонами. В отличие от параметров распределений, параметры процессоров сложны и требуют хорошего знания математического механизма, используемого конкретным процессором. Процессоры перечислены здесь только для того, чтобы читатель знал об их существовании, но подробно они в этой книге не рассматриваются.

Библиотека определяет также несколько типов, созданных на базе процессоров и адаптеров. Тип `default_random_engine` — это псевдоним типа для

одного из классов процессоров, параметризованных переменными, предназначенными для повышения эффективности использования. Библиотека определяет также несколько классов, являющихся полностью специализированными версиями процессора или адаптера. Ниже приведены процессоры и их специализации, определенные библиотекой.

Тип `default_random_engine`

Псевдоним типа для одного из процессоров, подходящего для большинства задач.

Тип `linear_congruential_engine`

`minstd_rand0` — имеет множитель 16807, модуль 2147483647 и приращение 0.

`minstd_rand` — имеет множитель 48271, модуль 2147483647 и приращение 0.

Тип `mersenne_twister_engine`

`mt19937` — 32-разрядный беззнаковый генератор вихря Мерсенна.

`mt19937_64` — 64-разрядный беззнаковый генератор вихря Мерсенна.

Тип `subtract_with_carry_engine`

`ranlux24_base` — 32-разрядный беззнаковый генератор вычитания с переносом.

`ranlux48_base` — 64-разрядный беззнаковый генератор вычитания с переносом.

Тип `discard_block_engine`

Адаптер процессора, отбрасывающий результаты базового процессора. Параметризуется базовым процессором для размера используемого блока и размера использованных блоков.

`ranlux24` — использует процессор `ranlux24_base` с размером блока 223 и размером использованных блоков 23.

`ranlux48` — использует процессор `ranlux48_base` с размером блока 389 и размером использованных блоков 11.

Тип `independent_bits_engine`

Адаптер процессора, генерирующий числа с заданным количеством битов. Параметризован базовым процессором для использования количества битов, генерируемых в его результатах, и целочисленным беззнаковым типом, используемым для содержания созданных битов. Определяемое количество битов должно быть меньше количества цифр, которое может содержать заданный беззнаковый тип.

Тип `shuffle_order_engine`

Адаптер процессора, возвращающий те же числа, что и его базовый процессор, но в другой последовательности. Параметризован базовым процессором и количеством переставляемых элементов.

`knuth_b` — использует процессор `minstd_rand0` с размером таблицы 256.